

Connect Intel — Product & Architecture Overview

Version: June 2026 · **Live:** <https://connectintel.net>
Purpose: Share with reviewers. Goal: HubSpot-grade revenue platform (CRM + Marketing + Team ops).

1. What Connect Intel Is

B2B revenue workspace: prospect, qualify, and close leads in one app.

Hub	What users do
CRM / Sales	Pipeline, lead records, deals, sequences, calendar, WhatsApp
Marketing	Campaigns, audiences, templates, automations, forms, analytics
Home / Analytics	Dashboard, team KPIs, activity log, my-day
Collaboration	Team chat (Chithi), notes, tasks
AI Prospecting	Search companies, save to pipeline
Workspace	Team invites, permissions, email domain, integrations

North star: HubSpot-style objects, timeline, permissions, reporting — fast SPA, no page reloads.

2. Roles & Data Scoping

Role	Pipeline access
Rep	Own leads only (owner_id = self) — cannot bypass
Manager	Team by default; “All Departments” → department scope
Org Admin	Full org pipeline + team settings, imports, permissions
Marketing roles	Manager / Executive / Read-only on marketing hub
Platform Admin	Connect Intel operator (customer admin, separate nav)

Scoping enforced in **Postgres** (pipeline_leads + COUNT RPCs), not in-memory scans.

3. Pages & Navigation (SPA)

Panel-based shell — URL drives panel state, not separate HTML pages.

/home/dashboard	→ Home overview
?panel=pipeline&status=new	→ CRM pipeline (filtered)
?panel=marketing	→ Marketing hub
?panel=team	→ Team & permissions
?lead=<id>	→ Lead record workspace

Layout: Sidebar (nav + scoped lead counts) · Top bar (search, ⌘K) · Panel content · Lead workspace overlay.

Key files: navConfig.js, AppShell.jsx, PipelinePanel.jsx, MarketingPanel.jsx

4. Code Structure

frontend/	React SPA (Vite) → site/
api/index.js	Single Vercel function → all /api/*
lib/server/	Auth, CRM, marketing, store, handlers/
supabase/	Postgres migrations
docs/	Blueprint, release, PRD

Handlers: CORS → requireUser() → business logic → JSON. ~100 API routes.

5. Data & Deployment

Hybrid storage

Layer	Storage	Use
JSON store	store_collections	Users, orgs, lead blobs, marketing metadata
SQL (hot path)	Supabase Postgres	Pipeline queries, RBAC, email queue, activity snapshots

Key tables: pipeline_leads, profiles, teams, departments, marketing_email_queue

Deploy flow

- 1. npm run prod:ship (build + checks)
- 2. git push origin main
- 3. GitHub CI + Vercel → https://connectintel.net
- 4. npm run prod:log marks live commit
- 5. Supabase migrations run manually in SQL editor

Flags: USE_PIPELINE_LEADS_TABLE, USE_PIPELINE_HIERARCHY_RBAC, USE_MARKETING_SQL_QUEUE

6. Feature Status (PRD)

Module	Live	Roadmap
Pipeline kanban / table / deals	✓	Multi-pipeline
Lead workspace + timeline	✓	—
Sequences & CRM rules	✓ partial	Unified automation engine
Marketing hub (campaigns, automations)	✓	CRM journey unification
Dashboard / team KPIs / my-day	✓	Custom report builder
Org admin (invites, permissions)	✓	Dept/team UI seeding
Team hierarchy RBAC	✓ SQL	—
Companies as parent object	partial	P0
Standalone deals + forecasting	partial	P0
Service tickets	partial	P2

7. HubSpot Gap — Review Priorities

- **P0:** Company-centric object model; standalone deal pipeline
- **P1:** One automation engine (CRM + Marketing); configurable reports

- **P2:** Service hub; multi-pipeline per product/team
- **P3:** Website tracking & behavioral scoring

Full blueprint: docs/CRM_PLATFORM_BLUEPRINT.md

8. Questions for Reviewers

1. Is rep / manager / org admin / marketing role split correct?
2. JSON + SQL hybrid until full Postgres cutover — acceptable?
3. Which P0/P1 gaps matter most for your workflow?
4. Panel-based SPA vs traditional routes — training concerns?
5. Priority: shared contact object + marketing/CRM journey view?